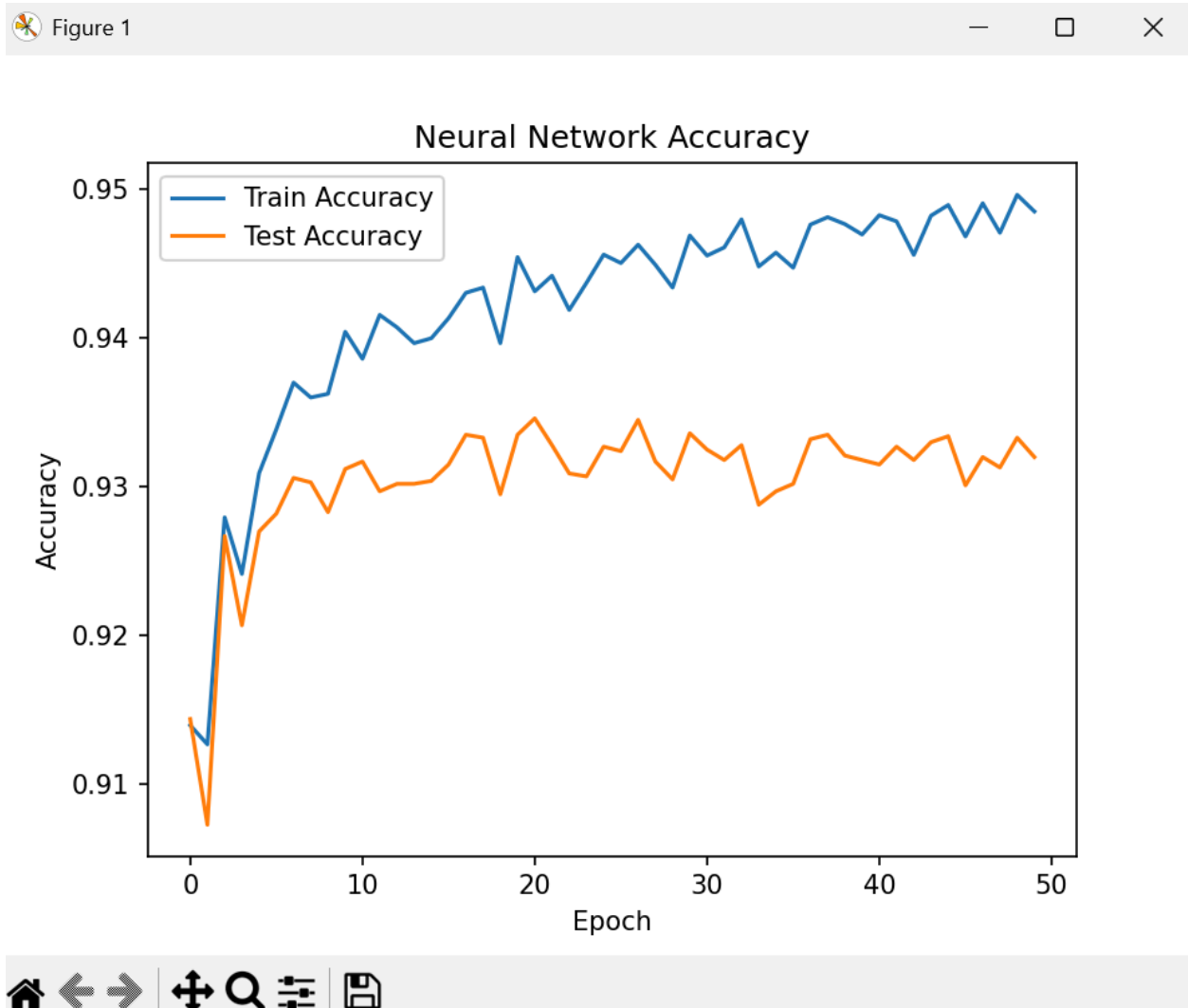


Experiment 1

Varying Hidden Units

I trained a two-layer neural network with different numbers of hidden units (20, 50, 100) while the rest of the parameters were consistent. The reason is to see how hidden units affect the accuracy.

N = 20



Confusion Matrix:

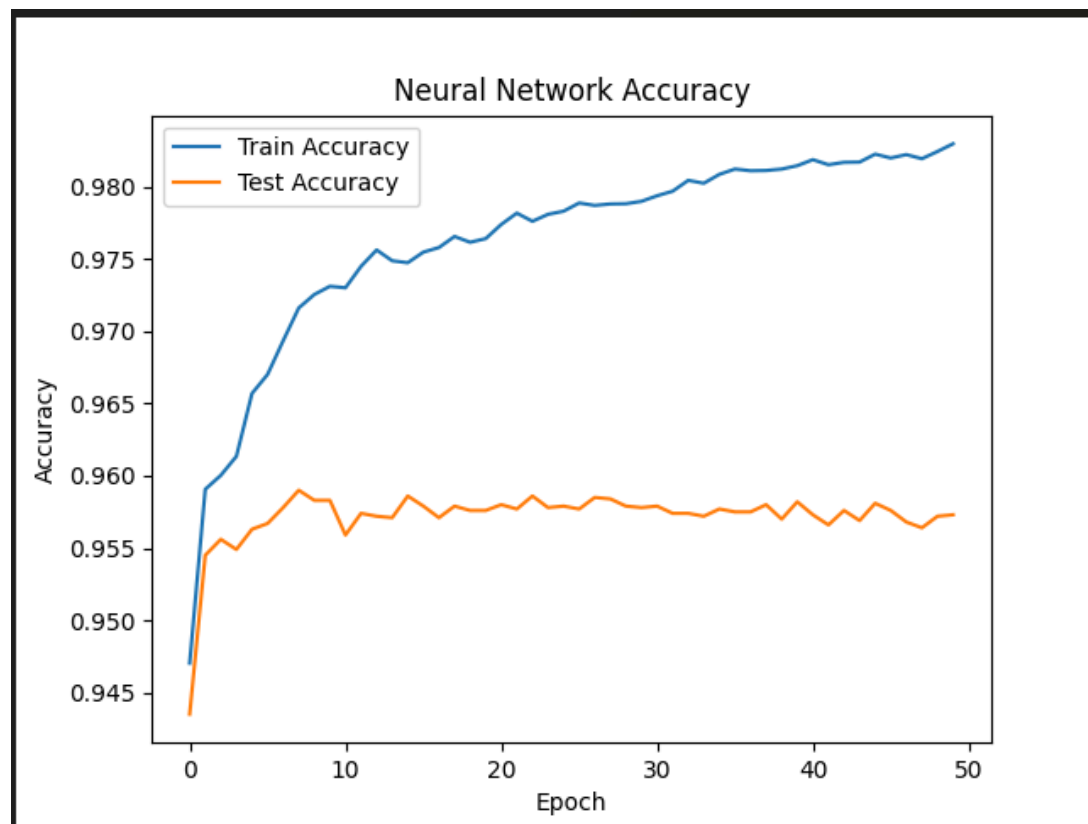
```
[[ 943  0 10  1  1  2  8  2 11  2]
 [  0 1120  2  2  0  1  3  0  7  0]
 [  2  4  925 18  9  0 16  7 44  7]
 [  2  1 13 924  0 26  1  6 25 12]
 [  0  2  2  1 886  0 15  2 10 64]
 [  1  4  4 20  0 803 15  4 29 12]
```

```
[ 7  6  2  0  3 11 899  3 26  1]
[ 0  9 19  4  4  2  1 950 11 28]
[ 5  2  2  6  6  9 11  3 921  9]
[ 2  6  3 15 13  3  0  6 12 949]]
```

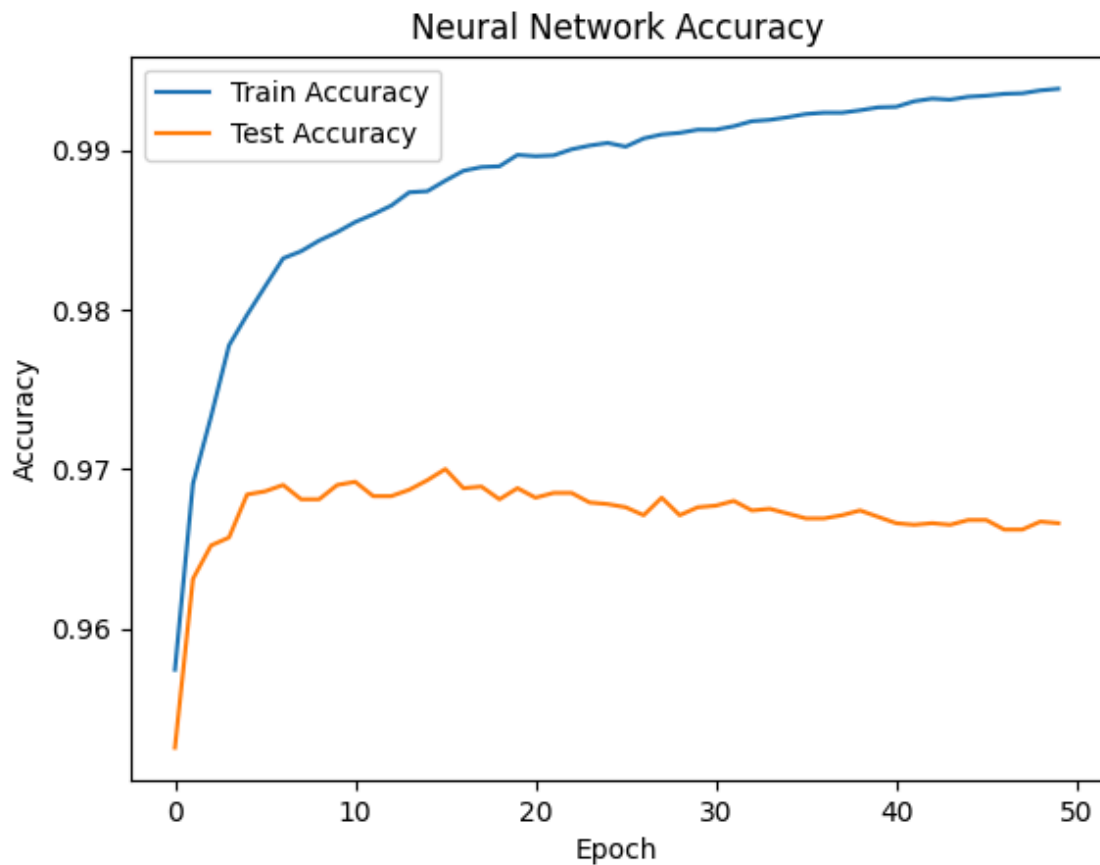
N = 50

Confusion Matrix:

```
[[ 965  0  1  3  0  2  1  2  5  1]
 [  0 1121  4  3  0  1  1  2  3  0]
 [  6  2 982 10  2  0  1  9 18  2]
 [  1  0 11 978  0  8  0  3  6  3]
 [  1  0  5  0 933  1  7  0  2 33]
 [  4  1  1 26  2 835  4  4 11  4]
 [ 11  5  7  1  4 11 910  0  9  0]
 [  1  7 14  8  2  1  0 970  5 20]
 [  4  1  5 11  7  4  4  4 925  9]
 [  3  6  1 14 14  1  0  5 11 954]]
```



N = 100



Confusion Matrix:

```
[[ 968  0  2  1  2  1  1  1  3  1]
 [ 1120  2  2  1  1  1  0  7  0]
 [ 1  2 997 10  2  1  1  5  9  4]
 [ 1  0  4 980  1  6  0  4 12  2]
 [ 3  0  2  1 943  0  4  3  2 24]
 [ 4  1  0 11  1 843 11  3 13  5]
 [ 4  3  2  1  2  9 927  0 10  0]
 [ 2  6 15  3  6  0  1 980  4 11]
 [ 5  0  3  4  5  3  5  2 945  2]
 [ 3  5  1 17  7  2  1  4  6 963]]
```

(1) How does the number of hidden units affect the final accuracy on the test data?

As I increased the number of hidden units, the overall classification accuracy improved. With the hidden units being 20 the confusion matrix showed more misclassifications through out multiple numbers. Telling me that it doesn't fully classify the numbers. With hidden units 50, it has significantly improved with more correct predictions. With hidden units, 100 showed the best performance with only a little bit of misclassification.

(2) How does it affect the number of epochs needed for training to Converge?

The network with a larger hidden size will reach higher accuracy levels. But on the other hand, it will take some more computation per epoch due to the increased number of weights. All of the models were set to 50 epochs.

(3) Is there evidence that any of your networks has overfit to the training data? If so, what is that evidence?

The only evidence that I found about overfitting is in the hidden units 100. Because the training accuracy continues to improve, and the test performance only shows very little gain compared to the hidden unit 50.

(4) How do your results compare to the results obtained by your Perceptron in HW 1?

The perceptron is limited to only learning in linear decision boundaries. While the neural network can learn a bit more complex patterns because of the hidden layers. This results in a higher accuracy and fewer errors.

Experiment 2

Varying Momentum

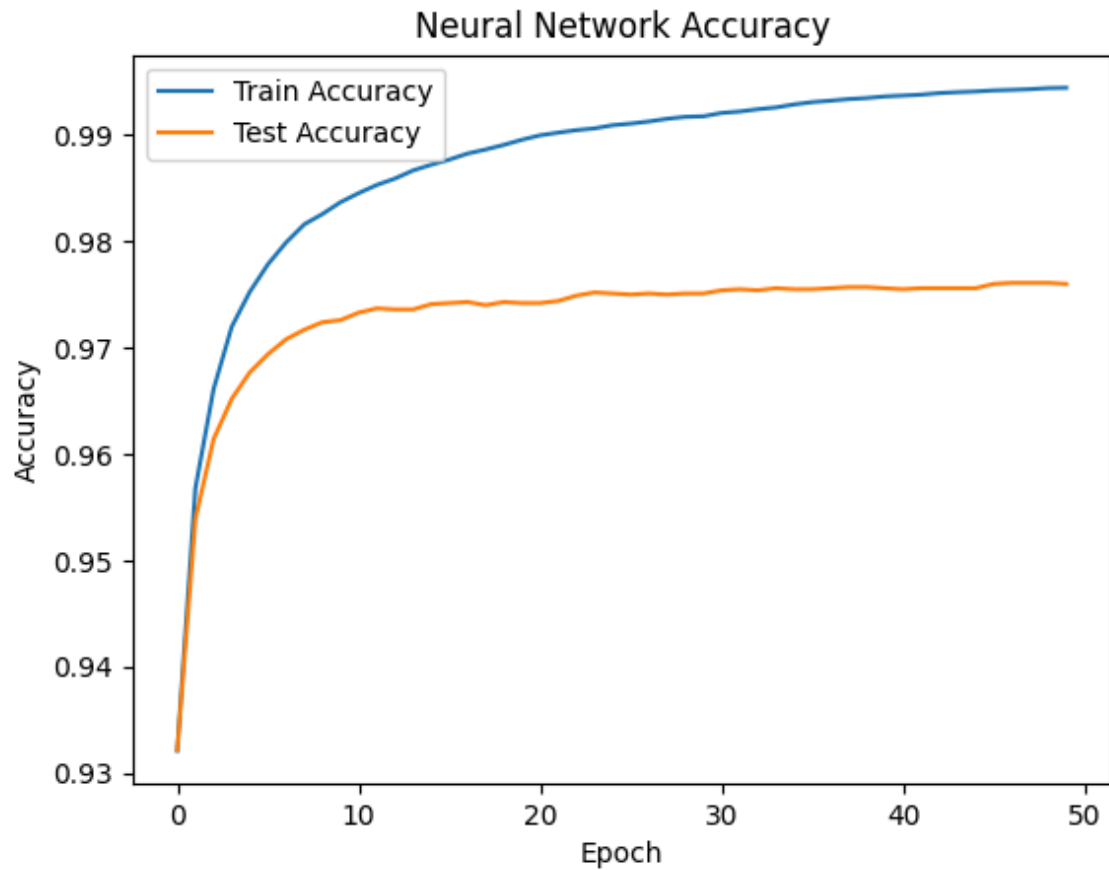
For this experiment, I had all of the parameters consistent only changing the momentum. To see how momentum affects learning behavior. The result showed that it had little effect on the outcome but helped smooth and slightly speed up convergence during training.

Momentum 0

Confusion Matrix:

```
[[ 973  0  0  0  0  1  2  1  3  0]
 [ 0 1124  4  1  0  1  3  0  2  0]
 [  4  2 1004  3  3  1  1  7  6  1]
 [  1  0  2 989  1  3  1  4  3  6]
```

```
[ 1  0  0  0 957  0  6  0  0 18]
[ 4  0  0 10  1 858  7  2  5  5]
[ 6  4  0  0  3  3 940  0  2  0]
[ 2  4 11  3  3  0  0 992  3 10]
[ 3  2  2  2  4  3  1  3 951  3]
[ 2  6  0  6  9  5  1  4  4 972]]
```

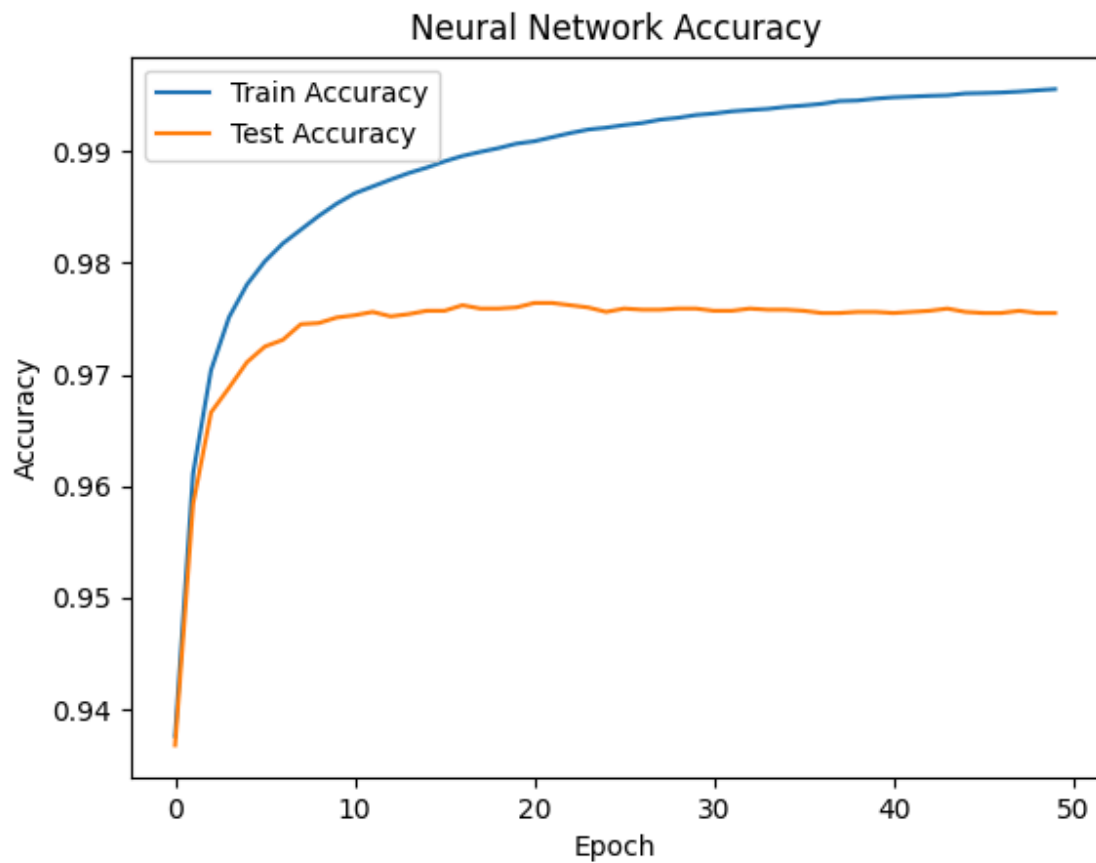


Momentum 0.25

Confusion Matrix:

```
[[ 970  1  0  0  0  2  3  1  2  1]
 [  11 25  1  4  0  1  2  0  2  0]
 [  2  2 10 07  5  2  1  4  4  1]
 [  0  0  3 99 5  0  5  0  2  3  2]
 [  2  0  1  1 95 6  0  7  1  0 14]]
```

```
[ 4 0 0 9 1 856 9 1 6 6]
[ 5 3 0 1 2 5 937 0 5 0]
[ 0 4 9 4 1 0 0 993 6 11]
[ 4 2 2 6 4 6 2 4 942 2]
[ 3 6 0 8 10 0 1 5 2 974]]
```

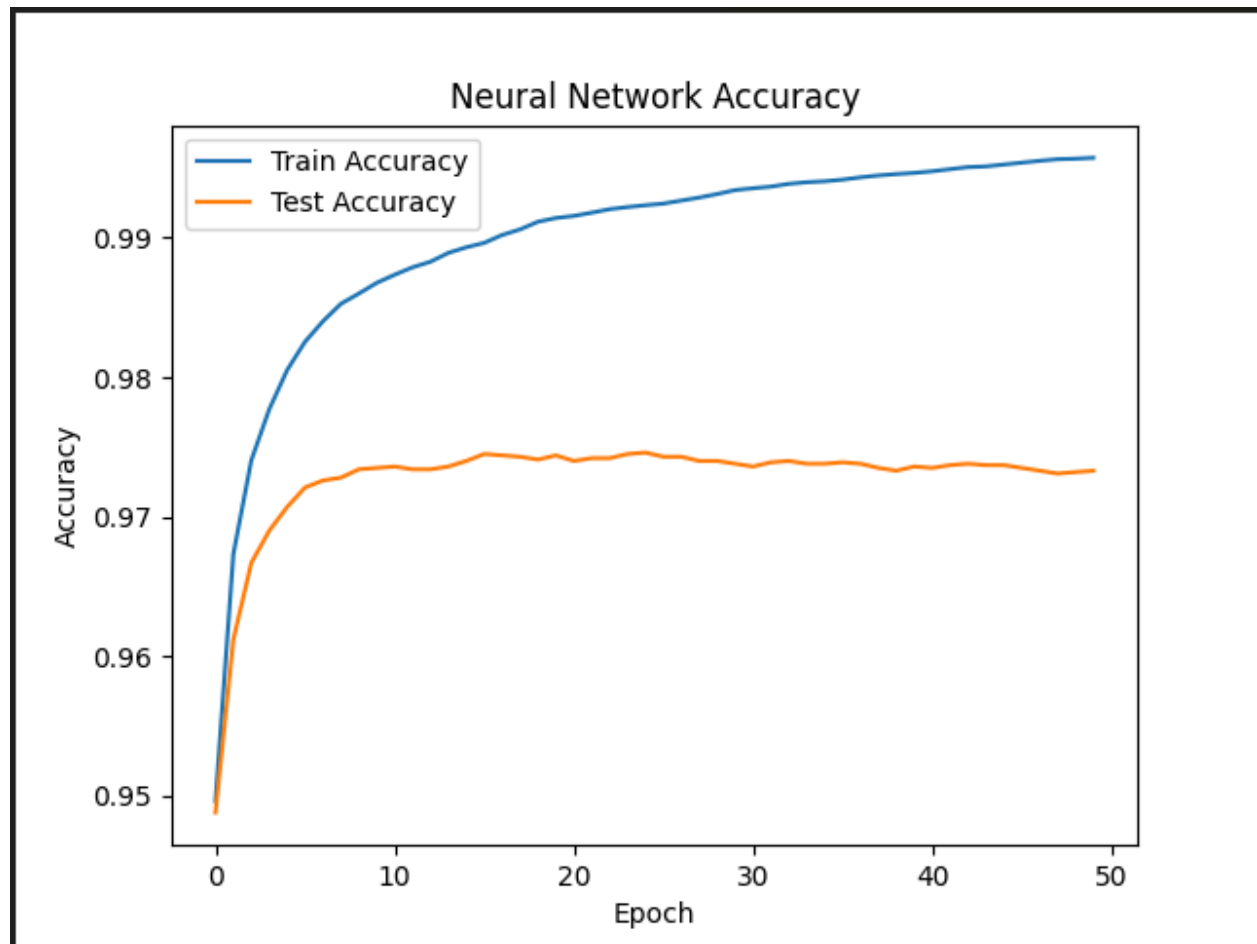


Momentum 0.5

Confusion Matrix:

```
[[ 970  0  1  1  0  3  1  1  2  1]
 [  11 25  0  4  0  1  2  2  1  0]
 [  7  3 995  8  2  0  5  4  8  0]
 [  0  1  4 988  0  4  0  5  3  5]
 [  1  0  3  0 959  0  5  0  1 13]
 [  5  1  2  7  1 853  8  1  7  7]
 [  6  3  0  0  2  6 933  0  8  0]
 [  1  5 12  2  2  0  0 990  3 13]]
```

```
[ 4  2  1  5  4  3  0  2 947  6]
[ 4  5  0  6 10  3  1  5  2 973]]
```



1) How does the momentum value affect the final accuracy on the test data?

Looking at the graphs across with momentum values. It looks like the accuracy is about the same across the different momentum values. Seems like it doesn't significantly improve the test performance. With higher momentum, it can decrease the accuracy slightly.

(2) How does it affect the number of epochs needed for training to Converge?

Momentum affects how quickly the network converges, not the accuracy. The higher momentum helps make the learning faster and smoother. Moderate momentum, the number of epochs is usually reduced to reach convergence. And the final accuracy remains nearly unchanged.

(3) Again, is there evidence that any of your networks has overfit

to the training data? If so, what is that evidence?

I don't see any overfitting. The training performs about the same with different numbers of momenta. It doesn't have a significant effect on the outcome.

Experiment 3

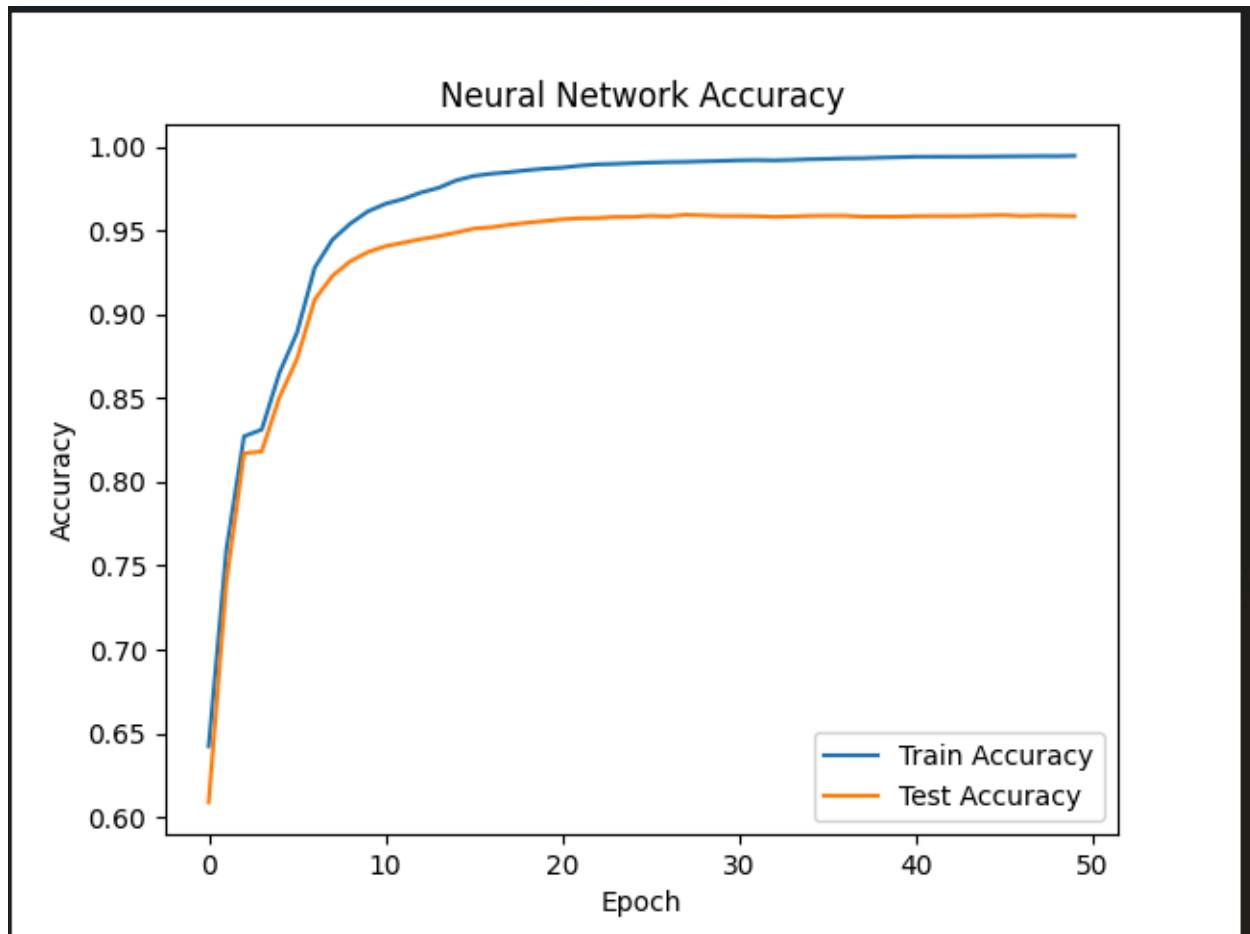
Varying Training Data Size

In this experiment, I trained the networks on different sizes of datasets. But still ensuring that each subset remained balanced across all classes. The goal was to see how the different sizes of the dataset affected the accuracy. Showing that the larger dataset has more accuracy and fewer misclassifications.

$\frac{1}{2}$ dataset

Confusion Matrix:

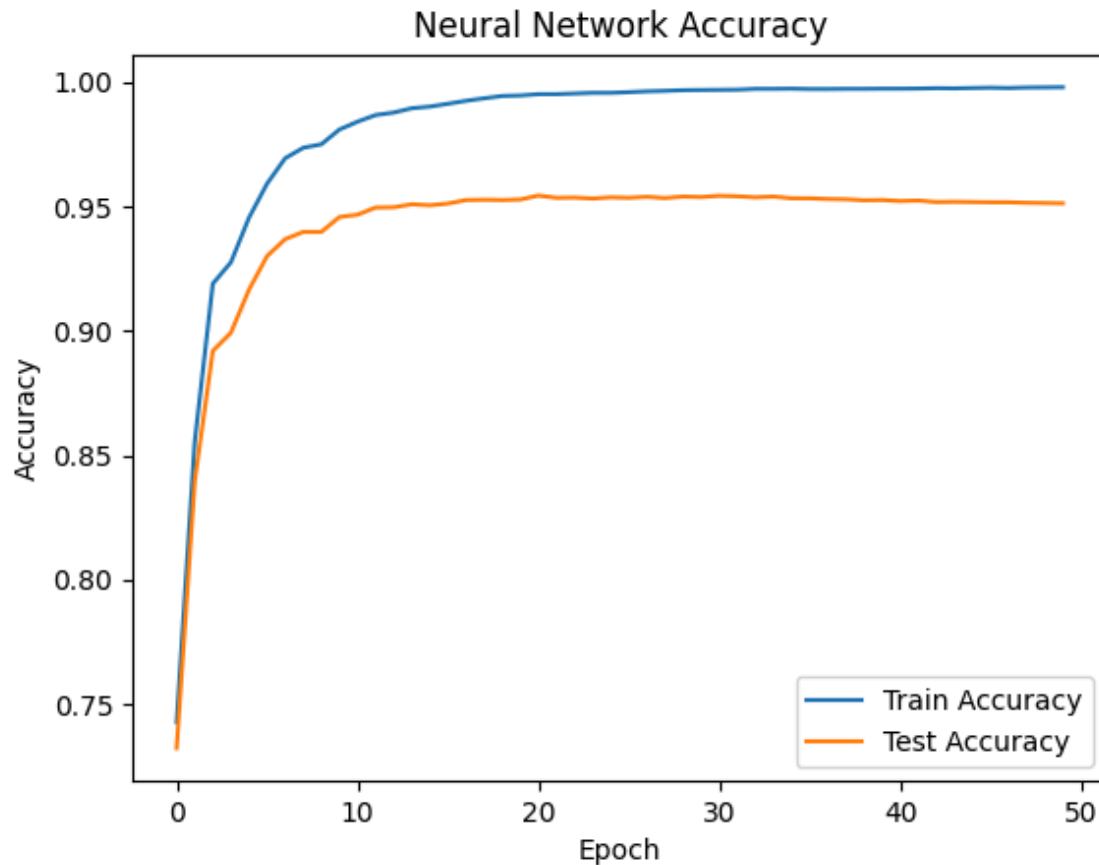
```
[[ 962   1   2   1   0   6   2   1   3   2]
 [ 11097   8   2   1   1   4   2  15   4]
 [ 22989   8   1   1   0   7  21   1]
 [ 117942   0  30   0   6  19   4]
 [ 104954   0   4   1   3  14]
 [ 30220864   7   1   9   4]
 [ 6210514914   1  13   2]
 [ 141951001965   7  16]
 [ 30443832945   2]
 [ 5203990721953]]
```



¼ dataset

Confusion Matrix:

```
[[ 952  1  4  2  1  5  3  1  5  6]
 [  11 110  6  1  1  4  1  1 10  0]
 [  2  0 989  2  3  4  1  7 22  2]
 [  4  0 14 925  0 22  0  4 36  5]
 [  1  0  5  1 935  1  6  0 10 23]
 [  2  0  3  7  0 852  3  2 21  2]
 [  7  2  9  0  6 16 900  0 17  1]
 [  0  6 31  1  3  2  1 966  6 12]
 [  5  0  4  2  4 10  4  3 938  4]
 [  2  4  1 11 16  6  0  5 18 946]]
```



1) How does the size of the training data affect the final accuracy on the test data?

Reducing the size of the training dataset negatively impacted the model's performance on the test data. The model with half of the dataset did better than with the quartered dataset. Also had fewer misclassifications. With less training data, the model has fewer examples that it can learn from.

(2) How does it affect the number of epochs needed for training to Converge?

The network trained on the smaller quarter dataset converged faster in terms of epochs. This is because it has less stuff to do. But faster convergence did not lead to better results.

(3) Again, is there evidence that any of your networks has overfit to the training data? If so, what is that evidence?

I think there is evidence of overfitting if we look at the quarter of the dataset. It has a limited training set, so it memorizes some patterns instead of generalizable features. Resulting in more misclassification.